Deadlock detection using MapReduce (PySpark)

CS3.304 Advanced Operating Systems, Assignment 5
Lakshyajeet Jalal, 2026201063

1. Running the code

The whole solution is one file. It takes the log path and the number of cores as arguments.

```
spark-submit AOS_Assignment5.py ../test_files/hpc_rag_1.log 4
```

It runs Spark in local mode with `local[N]`, where N is the core count. The input file, every shuffle and the edge partitioning all use N partitions, so the core count actually changes how much work runs in parallel. Versions used: Python 3.10.12, PySpark 3.4.1, Java 11.

2. Approach

2.1. Q1: building the graph

Each log line is split on whitespace. A `WAIT` line gives the edge P ⟶ R and a `HOLD` line gives R ⟶ P. A `reduceByKey` on the edge keeps one copy of each edge along with the earliest time it appeared, which Q3.1 needs later. Process and resource counts come from the distinct endpoints of the unique edges.

2.2. Q2: cycle detection

Before searching, I trim the graph. A node with no incoming edge or no outgoing edge can't be on a cycle, so every edge touching such a node is dropped. This repeats until the edge count stops changing. On large logs this matters a lot. My first version skipped it and ran out of memory on a 50,000 line log.

The search itself is MapReduce path propagation. Every process left after trimming starts a path of length one. In each round, paths are keyed by their last node and joined with the edge list (the shuffle), and the map step appends the next node. A path that comes back to its start is a cycle. A path that revisits any other node is dropped. The loop stops after 25 rounds, which is the maximum cycle length, or earlier if no paths are left.

A path is only extended to nodes greater than its start node. This means each cycle is found exactly once, from its smallest node, and is already in the rotation the output wants. Every process ID sorts before every resource ID, so that smallest node is always a process. Without this rule a cycle with k processes would be found k times and need a `distinct()` afterwards. The cycles are then sorted by length and then by name.

2.3. Q3.1 and Q4

For Q3.1, each cycle is split into its edges and joined with the first-seen timestamps from Q1. The latest of those is when the cycle was fully formed, and its date is counted per day. For Q4, in-degree and out-degree are counted on the unique edges, and `takeOrdered` picks the top five by degree, with ties broken by ID.

3. Results

3.1. Given log

This is the full output on `sample_hpc_rag.log` with 1 core. It matches a separate brute-force checker I wrote in plain Python, and the output is identical for 1, 2, 4 and 8 cores.

```
9
9
18
3.36
3
P101 R002 P106 R006
P105 R003 P107 R007
P102 R005 P103 R009 P108 R004
7
P101 P102 P103 P105 P106 P107 P108
2025-10-31 1
```

```
2025-11-02 2
R001 2
R008 2
R002 1
R003 1
R004 1
P104 2
P109 2
P101 1
P102 1
P103 1
```

There are 9 processes, 9 resources and 18 unique edges. Three deadlocks were found, two of length 4 and one of length 6, involving 7 processes. Resources R001 and R008 are the most contended, each requested by 2 processes, and P104 and P109 each wait on 2 resources.

3.2. Custom log

The given log is too small to say anything about scaling, so I generated `heavy_d10_w60.log` (50,000 lines). It contains a chain of 10 layers with 60 processes each. Every process waits on 3 resources held in the next layer, and deadlock cycles sit at both ends of the chain. Each resource has one holder and each node is on at most one cycle, as the assignment requires. Trimming can't remove the chain, so the search has to walk about 3.5 million paths in its deepest round.

On this log the program reports 776 processes, 663 resources and 4,779 edges. It finds 5 deadlocks of lengths 6, 8, 10, 12 and 16 involving 26 processes, formed on 4 different days. This also matches the brute-force checker for every core count.

4. Execution time (Q3.2)

I timed Q2 on both logs with 1, 2, 4 and 8 cores, on my laptop and on the ADA cluster. The laptop runs WSL2 with 16 logical CPUs and 15 GB of RAM. On ADA each run was a SLURM batch job on one `u22` node with 8 cores and 23 GB allocated. The timer covers trimming, the search and sorting the cycles.

Cores	Given, local	Given, ADA	Custom, local	Custom, ADA
1	3.36	7.28	329.89	682.82
2	3.61 (0.93×)	7.93 (0.92×)	184.38 (1.79×)	420.77 (1.62×)
4	3.81 (0.88×)	9.83 (0.74×)	152.81 (2.16×)	327.72 (2.08×)
8	5.30 (0.63×)	15.36 (0.47×)	128.90 (2.56×)	308.58 (2.21×)

Table 1: Q2 time in seconds, with speedup over 1 core in brackets.

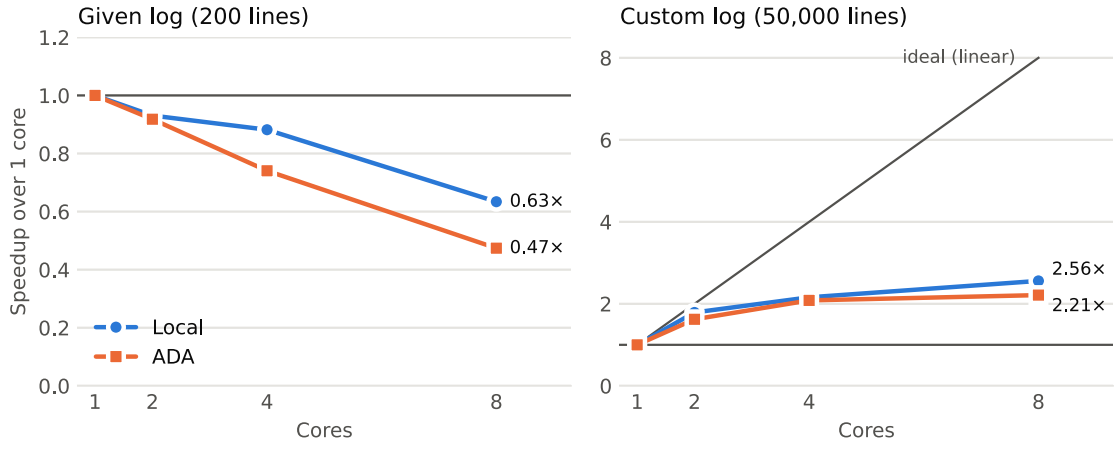


Figure 1: Speedup over 1 core. Below the 1.0 line, adding cores made Q2 slower.

5. Observations (Q3.3)

5.1. Given log

More cores made it slower, on both machines. After trimming, only the 14 edges of the three cycles are left, so each search round handles a few paths and the actual work takes microseconds. The time goes to Spark itself. Every stage has to schedule tasks, shuffle data for the join and pass it between the JVM and the Python workers. My code uses one partition per core, so 8 cores means 8 nearly empty tasks per stage instead of 1, and each one pays the same startup cost. Adding cores only adds overhead.

ADA is about twice as slow per core as my laptop, so the same overhead costs more seconds there and the slowdown is worse (0.47× at 8 cores).

5.2. Custom log

Here each round has hundreds of thousands of paths. The tasks do real work and the overhead becomes small next to it. Going from 1 to 2 cores almost halves the time. After that the gains shrink. Going from 4 to 8 cores saves 16% locally and only 6% on ADA.

This is Amdahl's law. Part of Q2 can't run in parallel. The rounds have to run one after another, each round ends by collecting results at the driver, and the trimming and early search rounds are too small to keep 8 cores busy. Fitting Amdahl's formula to the 8-core results gives a serial fraction of about 30% locally and 37% on ADA. That caps the speedup at roughly 3× no matter how many cores are added.

Spark's local mode adds contention on top of that. All workers share one JVM and its garbage collector, each core runs its own Python worker process, and they all compete for the same memory bandwidth. I think this is why ADA scales a little worse than my laptop. On ADA the 8 allocated cores have to run the 8 Spark threads plus the driver and the Python workers. My laptop has 16 logical CPUs, so 8 threads still leave spare capacity.

5.3. Conclusion

More cores only help when each task has enough work to outweigh Spark's fixed cost per task. For a log the size of the given one, 1 core is fastest. For the custom log, 2 to 4 cores give most of the gain and 8 add little.